

Tasks

Matrix Multiplication Profiling

In this lab, you will work with a provided implementation of matrix multiplication and explore how using an optimized math library and profiling tools can significantly improve performance.

The Matrix Multiplication

We have provided you with an implementation of the matrix multiply algorithm in C++. The program implements the matrix multiplication between matrix **A**start bold text, A, end bold text, a $m \times n$ m, times, n matrix, and matrix **B**start bold text, B, end bold text, a $n \times p$ n, times, p matrix, defined as:

$C = A \cdot B$ start bold text, C, space, =, space, A, space, B, end bold text

The resulting matrix **C**start bold text, C, end bold text is a $m \times p$ m, times, p matrix. Each element of matrix **C** is computed such as:

$$c_{ij} = \sum_{k=1}^n a_{ik} \cdot b_{kj}$$

You can review matrix multiplication in more detail here: [Matrix multiplication on Wikipedia](#)

Part 1: Leveraging an Optimized Math Library (OpenBLAS)

The BLAS (Basic Linear Algebra Subprograms) library provides highly optimized routines for vector and matrix operations. OpenBLAS is an open-source implementation of BLAS, tuned for modern processors.

The provided `matmul.cpp` contains three matrix multiplication implementations:

- JIK:** A naive nested loop implementation
- Built-in matrix multiply:** Using the array library's native operator
- BLAS implementation:** Started but incomplete

To complete Part 1, you will need to:

- Complete the BLAS implementation:** Open `matmul.cpp` and locate the `dgemm_matmul` function. Complete it using the BLAS routine:

- Update the shell script `matmul.sh`:** Modify the compilation command to include OpenBLAS. Ensure the `-lopenblas` flag is included so the linker uses the library.
- Compile and run your program**

Part 2: Profile Your Programs

Once you've completed the BLAS implementation, profile your program to analyze its performance.

- Enable profiling:** Update your compilation command in `matmul.sh` to include the profiling flag: `-pg`
- Analyze with `gprof`:** In `matmul.sh` include a command to use `gprof` to create a human-readable report and save it to `profile_report.txt`.

Submission

Submit `matmul.tar` containing:

- `matmul.cpp`
- `matmul.sh`

You can create the tar file with:

